

Usando ML para detectar fraude en tarjetas de crédito

Gary Briceño

- Ingeniero Electronico
- Universidad Nacional de Ingeniería - Perú
- Estudiante de la Maestría de IA
- Universidad de los Andes - Colombia

Enlaces:

- <https://www.linkedin.com/in/garybriceno/>
- <https://github.com/GaryBriceno>



¿Sabías que las pérdidas por fraude con tarjetas de crédito alcanzan más de **30 mil millones** de dólares al año a nivel mundial?

Hoy vamos a usar **Machine Learning** para detectar transacciones fraudulentas, usando un dataset real y técnicas clave como **balanceo de clases** y detección de **outliers**.

¿Por qué es un problema desafiante?

"Solo 0.17% de las transacciones en el dataset son fraude. Eso significa que un modelo podría tener 99% de precisión... y aún así fallar completamente."

Dataset: Credit Card Fraud Detection

Credit Card Fraud Detection

Anonymized credit card transactions labeled as fraudulent or genuine



[Data Card](#) [Code \(5446\)](#) [Discussion \(109\)](#) [Suggestions \(0\)](#)

About Dataset

Context

It is important that credit card companies are able to recognize fraudulent credit card transactions so that customers are not charged for items that they did not purchase.

Content

The dataset contains transactions made by credit cards in September 2013 by European cardholders.

This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

It contains only numerical input variables which are the result of a PCA transformation. Unfortunately, due to confidentiality issues, we cannot provide the original features and more background information about the data. Features V1, V2, ... V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise.

Given the class imbalance ratio, we recommend measuring the accuracy using the Area Under the Precision-Recall Curve (AUPRC). Confusion matrix accuracy is not meaningful for unbalanced classification.

Usability ⓘ

8.53

License

[Database: Open Database, Cont...](#)

Expected update frequency

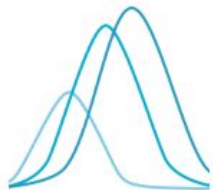
Not specified

Tags

Finance

Crime

Ref: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>



Reproducible Machine Learning for Credit Card Fraud detection - Practical handbook

🔍 Search this book...

Foreword

1. BOOK OVERVIEW

1. Book content and intended audience
2. Book contributions
3. How to use this book



Foreword

Payment card fraud is a major challenge for business owners, payment card issuers, and transactional services companies, causing every year substantial and growing financial losses. According to the 2019 Nilson Report, card fraud losses worldwide have increased from 9.84 billion dollars in 2011 to 27.85 billion dollars in 2018, and are projected to reach more than 40 billion dollars in 2027 [rep19].

Detecting fraud patterns in payment card transactions is known to be a very difficult problem. With the ever-growing amount of data generated by payment card transactions, it has become impossible for a human analyst to detect fraudulent patterns in transaction datasets, often characterized by a large number of samples, many dimensions, and online updates. As a result, the design of payment card fraud detection techniques has increasingly focused in the last decade on approaches based on machine learning (ML) techniques, that automate the process of identifying fraudulent patterns from large volumes of data [CLBC+19, DP15, PP19, SSB18].

The integration of ML techniques in payment card fraud detection systems has greatly improved their ability to more efficiently detect frauds, and assist payment processing intermediaries in identifying illicit transactions. Though in recent years the number of fraudulent transactions kept on increasing, the percentage of losses due to fraud started to decrease in 2016, a reverse trend that is associated with the increasing adoption of ML solutions [rep19]. On top of helping saving money, implementing ML-based fraud detection systems is today becoming a must-do for institutions and companies to gain the trust of their customers.

Ref: <https://fraud-detection-handbook.github.io/fraud-detection-handbook/Foreword.html>

PCA Transformation

- Por confidencialidad, el dataset solo contiene variables numericas, resultado de una **transformación PCA**.
- PCA transforma los datos originales en un nuevo conjunto de variables no **correlacionadas**, llamadas componentes principales.
- Se tienen dos características que no han sido transformadas:
 - Time: contiene el tiempo entre la primera transacción y cada transacción posterior en el dataset
 - Amount: Es el monto de la transacción
 - Class: es la variables respuesta, toma 1 en caso de fraude y 0 en caso contrario

Dataset

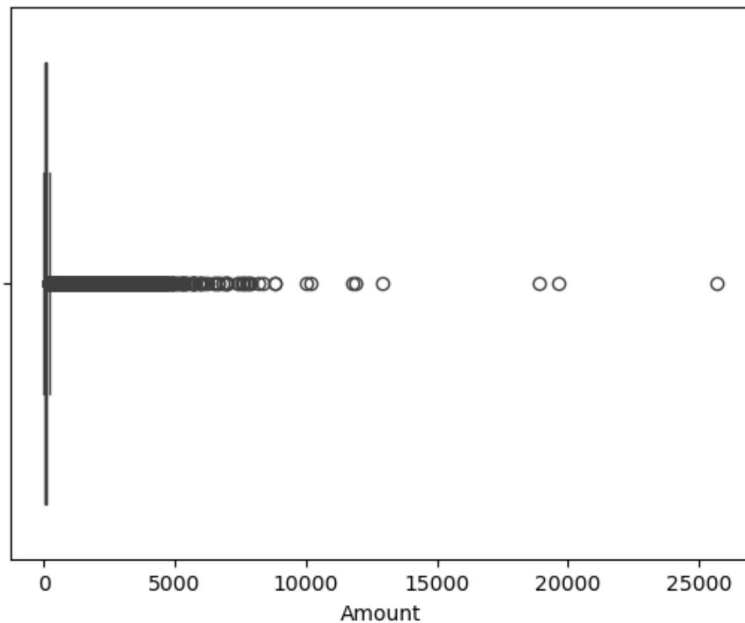
- Tenemos:
 - 31 Columnas, la mayoría anonimizadas (V1 a V28), adicional, tenemos “Time”, “Amount” y “Class”

V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
18321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
10018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
13198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
0309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
17193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0

Outliers: Valores atipicos

```
import seaborn as sns  
sns.boxplot(x=df['Amount'])
```

<Axes: xlabel='Amount'>



Aplicar SMOTE

```
# Distribución de clases  
df['Class'].value_counts(normalize=True) * 100
```

```
Class  
0    99.827251  
1     0.172749  
Name: proportion, dtype: float64
```

- SMOTE : Synthetic minority Over-sampling Technique
- Se usa para balancear datasets desbalanceados, creando data sintetica.
- Es compatible con muchos clasificadores, como RandomForest, SVM, etc.
- Mejora el aprendizaje al repetir la clase minoritaria

Ref: https://imbalanced-learn.org/stable/over_sampling.html#smote-adasyn

Gracias